Git practical

Nick Brown

6 October 2016

1 Introduction

These practicals are designed to give some experience of using Git by going through the steps required to setup and use a Git instance on your local machine. Often Git is used remotely (i.e. the repository is based on a remote machine and you use Git to upload and download data from it) but it can also be used locally (i.e. with the repository on the same machine that you are working on) and this mode of using Git is useful for experimenting with, and learning, its functions and operations.

2 Tell Git who you are

On Cirrus you need to tell Git who you are, this is a good idea regardless as it makes it clearer who changed what in the history:

```
$ git config --global user.name "My Name"

$ git config --global user.email myuun@sms.ed.ac.uk
```

3 Create the master repository

The first thing that you will need to do is to create a master Git repository. Creating a repository is based around the `git` command as follows:

```
$ cd $HOME

$ mkdir mastermyproject.git

$ cd mastermyproject.git

$ git --bare init
```

Note that the location of your repository can be anywhere (as long as you have write permissions) and the name can be anything but the postfix `.git` is conventional.

4 Create your local repository

The basic pattern adopted by Git is to have a single master repository and then each user has a representative local (private) repository into which they make their changes. There is an important concept here; the local repository is separate from your physical files under source control. Therefore any changes to the actual files themselves will need to be explicitly communicated to the local repository (which stores a representation of these along with their history in compressed binary format.)

To create your local repository you also use the `git` command as follows:

```
$ cd ~

$ mkdir myproject

$ cd myproject

$ git init

$ echo "This is a test" > test

$ echo "Hello world" > afile

$ git add *

$ git commit –m "Initial commit of my project"
```

The `git init` command will set up the local repository and actually creates a hidden `.git` directory which contains the internal workings of the repository itself. For purposes of demonstration we next create two example files `test` and `afile` which will form the initial content of our repository. In reality a more common use case is to cd into an existing directory full of files that you wish to place under source control and initalise the repository from there.

The next stage is to add all of our files within the chosen directory into the Git local repository which is a two stage process. The first stage comprises of telling Git which files to actually add via the `git add` command. This will `stage' all files inside that directory and sub directories. The second action we need to perform is to actually commit these files into the local repository which is done via `git commit`. In our example we also supply an associated commit message.

5 Connecting the local and remote repository

So, let's have a think what has achieved up to this point; we have created our master repository, have set up a local repository and checked in (committed) some files into this.

The next job is to connect the two repositories together which basically amounts to telling your local one the location of the master repository. Ensure that you are inside your local repository directory as above and then issue the commands:

```
$ git remote add origin file://$HOME/mastermyproject.git

$ git push –u origin master
```

The first command configures our local repository to use the location of the master repository and name this connection `origin`, which is convention. The `push` command will push the newly committed files in your local repository, and specifically the `master` branch, to the remote repository named `origin`. If you specified the path directory (rather than using $HOME) and get an error due to the capitalisation of `/Home` on the Computational Physics Lab machines, then issue `git remote rm origin` and then redo the above but capitalising the H in `home`.

6 Modifying files

Now that we have set up and connected our repositories, in normal operation you just need `git add` to stage the required files, `git commit` to commit these files into your local repository and then issue the `git push` command to update the remote repository with changes in your local one. To illustrate this we are going to modify one of our existing files and explore how we can use Git to manage this.

```
$ echo "SOME MORE TEXT" >> test
```

Via the above we have added some text into our `test` file. Now issue the `git status` command and you will see that Git tells us that this file has been modified with respect to the local repository. Want more information? Simply issue `git diff test` to find out exactly what has been changed in that file. So let's imagine that we are happy with our changes; the next step is to check that modified file in and push the changes into the remote repository:

```
$ git add test

$ git commit -m "Changes made to the test file"

$ git push
```

Now if we reissue the `git status` command then we shall see that there is nothing new to commit (because we have just committed all changes.) Here is a hint: Sometimes the `git add` command can get quite tiring, especially if you have made many changes. Instead, you can simply do away with the `git add` and append the `a` option to the commit command which instructs Git to commit all changes, therefore the above commit command would change to `git commit -am "Changes made to the test file"`.

7 Adding files

Adding new files is in many ways similar to modifying files. However note that you must issue the `git add` command to explicitly stage these files before committing them.

```
$ echo "A new file" > newfile
```

In the above we have created a new file with some text in it, now issue the `git status` command. If you look at the output carefully you will see that `newfile` is listed however the command reports that this is untracked. Untracked basically means that the file exists however is not being managed by your local repository. To track the file we simply stage it by issuing the `git add newfile` command. Once you have done that, let's have a look at the output of `git status` again … see that? Git is now reporting that it knows about this file and is ready to accept it and send it to the remote repository via the (hopefully familiar by now) commands:

```
$ git commit -m "Added a new file into my repository"
```

```
$ git push
```

8 Moving or removing files

Git is actually quite good at identifying when managed files have been removed or modified. However Git, like SVN, provides explicit commands to perform this functionality and for safety it is always best to use these. In this section we are going to move one file and remove another for you to discover how this is best achieved:

```
$ git rm afile

$ git mv newfile bfile

$ git commit -am "Removed file and renamed another"

$ git push
```

9 Reverting changes

It happens quite often - you start making changes to a file and then realise that actually, what you have done is wrong, and you need to go back. Git provides a number of facilities to support this, we will just look at one of these with a simple exercise.

```
$ echo "A temporary change" >> test

$ git diff test
```

When issuing the diff command we can see that Git reports some differences between the working copy of this file and the version stored within our local repository. Next issue `git checkout test` which will "roll back" our uncommitted changes to the version stored within the local repository. If you reissue the `git diff test` command you will see there are no differences now reported between the working copy of this file and the version committed. Top tip: to revert all your current changes, execute `git checkout -f`.

10 Viewing the history

As with the vast majority of revision control systems, Git keeps a record of all commits and changes to files. This information is available via the `git log` command. Without any further parameters the command provides a summary of all commits and their messages. For more detailed information about commits you can use `git log --stat` or `git log -p`. The log command also allows you to see changes to specific files, issue `git log -- test` to view all commits which have changed the file named `test`. As with log information about commits you can view more detailed file history using the `--stat` or `-p` options too.

11 Multiple machines

For this next exercise, imagine this common scenario; we have been happily coding away for some time using our own local machine with a local Git repository and some remote repository too. Suddenly we get to the stage where we want to compile, debug and run our code on ARCHER. The naive approach would be to simply copy the required files across, but we can do better than that!

Instead a local Git repository can be created on the ARCHER login node which points to our remote master repository. This local repository can then be used to pull down the current code version and what's more, any changes to our code on ARCHER can then be pushed back into the remote repository.

For the purposes of this practical we are going to create another local repository on the same machine that holds our existing local and master repositories (just to avoid everyone logging into ARCHER and clogging up that node.)

```
$ cd ~

$ git clone file://$HOME/mastermyproject.git
```

This will create a `mastermyproject` directory and place the existing repository contents within it. We can then use that local repository as we would any other. Now that we have two local repositories and one master, this brings us onto the `git pull` command, which pulls down changes from the master repository into the local one. Below is an example to illustrate how this works:

```
$ cd mastermyproject

$ echo "More text" >> test

$ git commit -am "A test of more changes"

$ git push

$ cd ../myproject

$ git log -1

$ git pull

$ git log -1
```

After modifying a file in our newly cloned directory we commit it to the local repository and push the changes to the master. Then we go into our other local repository and issue the `git log -1` command, which simply displays the most recent commit (that it knows about.) As you can see at this stage the commit we just performed from our other local repository is not listed. Once the `git pull` has been issued then this local repository will pull down the latest changes from the master and a further look at the log contains them.

12 Resolving Conflicts

Continuing our scenario in the previous section, imagine you have modified the same file both on ARCHER and your own local machine and want to check in the changes made in both versions. Due to the distributed nature of Git it has some very powerful mechanisms for managing merge conflicts, we are just going to scratch the surface here. Starting from our `myproject` local repository:

```
$ echo "Some changes" >> test

$ git commit -am "Some initial changes"

$ git push

$ cd ../mastermyproject
```

```
$ echo "This change will conflict" >> test

$ git commit -am "A test of conflicting changes"

$ git push
```

The last push will give us a merge conflict, which will look something like:

```
To file:///lustre/home/d136/myusername/mastermyproject.git

 ! [rejected]        master -> master (fetch first)

error: failed to push some refs to 'file:///lustre/home/d136/myusername
/mastermyproject.git'

hint: Updates were rejected because the remote contains work that you do
not have locally. This is usually caused by another repository pushing to
the same ref. You may want to first merge the remote changes (e.g.,'git
pull') before pushing again.

hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Firstly let's issue a `git pull` to grab the latest version from the master repository. Whilst Git is quite clever and will attempt to sort out any conflicts itself at this point, if there is any doubt it will require the user to manually fix the issue. If there are merge conflicts that cannot be sorted automatically then a message is displayed and the offending files are annotated as in this case. Here Git has annotated the `test` file illustrating exactly what the conflict is, which are between <<<<<<< and >>>>>>> markers. Edit this file to resolve the conflict by removing this marker lines and any duplicated text and then issue the following to commit this into your local repository and push the changes to master.

```
$ git add test

$ git commit -am "Fix for merge conflict"

$ git push
```

13 Repository hosting

There are numerous free Git repository hosting websites which researchers often find useful for coursework and especially their dissertations. A list of common ones can be found at https://git.wiki.kernel.org/index.php/GitHosting with BitBucket and GitHub being the most popular.

14 Summary and further information

This practical has given an overview to the common functionality provided by Git which you might find useful. Whilst we have used the local file system to host our master Git repository, in reality this can be hosted in many different manners including networked file systems, SSH (my preferred option), Apache and the Git deamon.

The Git man pages are very informative, additionally, if you issue the `git help [command]` (where `[command]`) is replaced with whatever command you want more information about, then the specific man page for that command will be displayed. Another recommended source for further information can be found

at http://git-scm.com/documentation which not only recaps the basics discussed here but also illustrates more complex aspects such as branching, filtering out specific files, tagging and how to customise the configuration for your own requirements.

Software Carpentry provide an introduction to Git: Daisie Huang and Ivan Gonzalez (eds): "Software Carpentry: Version Control with Git". Version 2016.06, June 2016, https://github.com/swcarpentry/git-novice, 10.5281/zenodo.57467.

Eric Sink's Version control by example provides an acclaimed introduction to basic concepts, Subversion and Git: Sink, E. Version Control by Example. Pyrenean Gold Press, 25 July 2011. ISBN-10: 0983507902, ISBN-13: 978-0983507901. https://www.amazon.com/Version-Control-Example-Eric-Sink/dp/0983507902/156-0246549-6231041. Online at: http://www.ericsink.com/vcbe/.

Check out the visual Git reference for pictorial representations of what Git commands do (http://marklodato.github.io/visual-git-guide/index-en.html).

A last piece of advice, if you are going to use Git, then install `git gui` on your local machine. This is a very simple and lightweight GUI that, combined with the command line as discussed in this practical, simplifies managing Git repositories.